Semi-Supervised Labels and Outcome Prediction for Clinical Trials

CS5785 / ORIE5750 / ECE5414 — Midterm Project

Anirudha Hadagali
Cornell University

Michael Mazzola
Cornell University

Tarun Mandalapu
Cornell University

Abstract

We address two clinical trial prediction tasks. In Part 1, only 157 of 1,171 training trials have observed stopping-reason labels. Using 150-dimensional embeddings, we combine an anchored self-training classifier with a second pipeline that adds graph, cluster, and neighbor information. Our fixed 80/20 blend achieved a Kaggle sample-wise F1 of 0.89935, compared with 0.89636 for anchored alone.

In Part 2, we predict whether a trial meets its primary endpoint. To account for the later test period, we give recent trials more weight and incorporate neighbors that favor recent matches. Blending in 30% of an earlier text-and-tree ensemble achieved a Kaggle log loss of 0.58465. A later blend improved our historical evaluation scores but scored 0.58492 on Kaggle, so we retained the earlier model.

Team responsibilities. All three authors worked on both parts and on this report. **Michael Mazzola** led the Part 1 model. **Tarun Mandalapu** built the first Part 2 classifier, and **Anirudha Hadagali** led the later Part 2 tuning. Tarun Mandalapu also drafted the project notes used to prepare this report.

1. Methods

1.1. Part 1: stopping reasons

1.1.1. Data and evaluation

The dataset contains 1,561 trials that stopped early [3, 11]. The training set has 1,171 trials: 157 with observed labels and 1,014 with every reason marked -100, indicating an unknown label. The validation set has 78 labeled trials, and the test set has 312 later trials from 2010 to 2016. A trial can have more than one of five stopping reasons: Safety, Funding, Efficacy, Recruitment, and Other. Among the 235 labeled trials, Recruitment appears on 64% and Funding on 5%. Most labeled trials are from earlier years, so these rates may differ in the test set.

We initially selected models and features using sample-wise F1 on the supplied validation set, alongside per-reason F1 and its macro average. As we tested more configurations, we became concerned that repeatedly tuning against the same 78 trials could overfit our model choices to that particular split. A single trial can change sample-wise F1 by about 1.3 percentage points. This is selection overfitting, even when validation labels are excluded from training.

We therefore moved to repeated grouped cross-validation as a more stable guide for tuning, averaging over eight repeats of five folds on both the 157 training labels and all 235 labels, called Train157 and Pooled235 in the notebooks and results. This lets us judge whether a change helps across several held-out groups rather than only the original validation set. Trials with identical embeddings stay together, and we remove masked duplicates of held-out trials. We also use three chronological windows totaling 98 predictions to check performance on later trials. These reused development splits reduce dependence on a single holdout, but do not eliminate selection bias or provide a new independent test set.

1.1.2. Models and training

We use the 150 supplied stopping-reason embedding coordinates as features. Trial IDs align rows, and start years define our chronological evaluation splits. In an earlier self-training model, adding start year, eligibility text, disease, or drug features [5, 6, 9] did not improve validation sample-wise F1. Embeddings alone scored .893, compared with .891 with year, .876 with eligibility or disease text, and .889 with fingerprints. These experiments guided our feature choices, although we did not repeat the same feature comparisons with the final blend.

Our *anchored* component standardizes embeddings using the labeled and masked training rows. A 30-component diagonal Gaussian mixture uses 30 principal components [2, 7], with k-means initialization, three restarts, and seed zero. EM stops when the average lower-bound gain falls below 10^{-3}, with a maximum of 100 iterations; we check convergence before using the fit. The logistic classifier allows 3,000 iterations with tolerance 10^{-4}. For each component,

we estimate reason frequencies from the current training labels, adding one pseudo-count based on the overall rates. The pseudo-label teacher mixes 5% of these profiles with 95% of the classifier score. Our earlier k-means and DBSCAN models performed worse on Kaggle. DBSCAN-plus-logistic reached .910 validation F1 under an earlier protocol, but its original and 235-label submissions scored .87500 and .86452 respectively.

The anchored classifier combines multiclass logistic regression ($C = .1$), cosine 10-nearest-neighbor votes, and shrinkage linear discriminant analysis (LDA), with fixed weights .5/.3/.2. All use the 150 standardized coordinates. A multi-label trial is duplicated across its positive classes for the multiclass fits. Logistic weights divide its mass across positives and use class weights fixed from observed labels. Neighbors vote for every positive label, weighted by nonnegative cosine similarity squared. LDA uses equal class priors.

Two pseudo-label refreshes give three classifier fits in total [10, 13]. We accept a masked trial when its highest teacher score is at least .8. At each refresh, we rebuild the training set from observed labels and the currently accepted pseudo-labels. These pseudo-labeled rows receive logistic weight .25; the neighbor and LDA components use them without that downweighting. The final 235-label anchored fit accepts 757 masked rows in the last refresh. We keep observed labels fixed throughout self-training.

Our graph-based label-combination model, called *Cand2* in the notebooks, models eight fixed label combinations. It adds soft eight-cluster k-means membership and 20-neighbor label-vote features to standardized embeddings. Its Correct-and-Smooth labeler [4] uses a symmetric 10-neighbor graph on scaled, L2-normalized 64-dimensional PCA coordinates. Correction and smoothing coefficients are .8 and .419962. Generated labels require confidence .5 and receive weight .0630725 in balanced, weighted shrinkage LDA. To prevent leakage during evaluation, we refit preprocessing, graph labeling, and calibration within inner grouped partitions and remove duplicates of held-out trials. We fit temperature calibration using the inner held-out predictions. We average five seeds and convert combination probabilities into five reason marginals.

The selected rule combines anchored raw scores and Cand2 marginals:

$$s = .8s_{\text{anchored}} + .2p_{\text{Cand2}}.$$

We output every reason with $s_j \geq .3$ and always include the highest-scoring reason. We treat these scores as decision scores rather than calibrated probabilities and keep the blend weights fixed. For standalone Cand2, we instead decode label combinations by maximizing expected F1.

Beyond these components, we compared known-label logistic regression, nearest-neighbor classifiers, Gaussian naive Bayes, RBF support vector machines, and LDA ensembles. We tested k-means, GMM, DBSCAN, and graph-based label generation, then explored kernel and symbolic-regression (PySR [1]) blend partners, learned combiners, and alternative treatment of secondary Other labels. These comparisons cover different ways to generate labels, represent trials, and combine predictions; the notebooks retain the full configurations.

We fit both components using training inputs only and use test embeddings only for prediction. For our final submission, we refit anchored on all 235 observed labels plus the 1,014 masked rows and combine its scores with the saved five-seed Cand2 predictions. The .891 validation score comes from a separate fit using only the 157 training labels; it does not independently evaluate the final model trained on all 235 labels. Our final-model notebook documents both components and the complete blend. For local blend comparisons, we combine the components' saved held-out scores on the same trials, without retraining them for each set of blend weights.

1.2. Part 2: trial outcomes

1.2.1. Data and evaluation

Each row represents one trial in one phase. Combined-phase trials appear twice, so we keep their rows together in every split. The training set has 8,037 rows through 2013, validation has 1,013 rows from the same years, and the test set has 1,500 later rows from 2013 to 2022, with 500 per phase. We evaluate models using the average of the three phase log losses, matching the Kaggle metric. We also examine F1 at a 0.5 cutoff, ROC-AUC, and precision-recall AUC, computed as average precision with success as the positive class. We join Part 1 stopping reasons after prediction to analyze errors; neither reason labels nor reason embeddings enter the outcome models.

1.2.2. Models and training

Our baseline uses logistic regression on eligibility text: TF-IDF features from words and bigrams, at most 35,000 terms, minimum document frequency 3, sublinear term frequency, three phase indicators, and $C = 2$. Separate phase-only fits overfit Phase I. Drug fingerprints did not reliably help, and a matched drug is less common in test (39%) than in training (46%). Naive Bayes ranked trials reasonably well but produced overconfident probabilities, resulting in poor log loss.

We then built on a stack of eight classifiers [12]: logistic models on eligibility text, diseases, drug names, codes, drug annotations, and metadata, plus naive Bayes and a 50-neighbor model on the eligibility text. A second logistic regression learns to combine them from five-fold out-of-fold predictions, keeping each trial within a single fold. Recent trials get more weight, with a three-year exponential time

constant, so a trial loses about half its weight every two years. That stack, refit on all 9,050 labeled rows, scored 0.58688. The same stack without recency weights scored 0.58810. Our first ensembles had used only the 8,037 training rows, so part of the earlier gap was the extra data.

The neighbor model within the stack ignored time and pulled estimates toward a 50% success rate. To address this, we created a recency-and-neighbor blend (submission file 10 in the notebooks) by averaging the recency stack with a second stack that uses a shorter, two-year time constant. The second stack also includes eight neighborhood estimates that vary the number of neighbors and similarity weighting, with some incorporating disease, drug, and ICD text. We give the two stacks equal weight. This improved every chronological window and achieved 0.58609 on Kaggle, although log loss on the random validation split became slightly worse.

File 10 performed less well on splits that mixed years, while our earlier text-and-tree ensemble was weaker on later trials. Its trees use CatBoost [8], with 1,000 boosting iterations, learning rate .035, and L2 leaf regularization 6. The earlier ensemble assigns half its weight to three logistic models, a quarter to a depth-6 tree, and a quarter to a depth-8 tree. Combining the models helped: a 20% contribution from the earlier ensemble achieved 0.58474 on Kaggle. Increasing its share to 30% reduced log loss by a further 0.00009. We selected this blend, file 13, with submitted probability

$$0.70\,P_{10}+0.30\,P_{\text{early}}, \quad P_{10}=0.50\,P_{\text{recency}}+0.50\,P_{\text{fast neighbors}}.$$

We clip the final probabilities to $[.001, .999]$. After fixing the model configuration, we refit the supervised components on all 9,050 labeled rows. The original stack allows vocabulary construction to use unlabeled test text, but no test labels are used in training. A later blend (file 15) with an additional tree and a phase trend improved our local scores but achieved a worse Kaggle log loss of 0.58492, so we retained file 13.

1.2.3. Validation and error analysis

For later outcome candidates, we compare the supplied validation split, two repeats of three-fold trial-grouped cross-validation, and three forward windows trained through 2007, 2009, or 2011. The grouped checks compare models across held-out trials, while the forward windows assess the shift to later test years. We refit preprocessing within each training split, including inner stacking folds, and track the worst temporal window as well as the mean. All official validation metrics below hold out the 1,013 validation outcomes from fitting, even though we later refit the submission model on all 9,050 labeled rows.

For failed validation trials, we match trial IDs to Part 1. We use observed training or validation reason labels where available; otherwise we use predictions from our frozen historical DBSCAN/logistic model, with a 0.5 cutoff and highest-score fallback. We keep this reason assignment fixed across outcome models. Unmatched trials form the "no recorded stopping reason" group. Multi-label trials enter each applicable category, and a combined-phase trial is counted once within each phase. The false-success rate is the fraction of failed trials assigned success probability at least 0.5.

2. Results

2.1. Part 1: stopping reasons

Table 1 compares our Part 1 models on the same grouped splits. We tested small contributions from LDA or Cand2 while keeping anchored as the main component. Cand2 was weaker on its own, but its graph and label-combination approach could complement anchored's predictions. The 80/20 blend matched anchored's original validation F1 and remained close on both CV pools, with small gains in their means and the pooled chronological score.

We were concerned that anchored might not generalize as well to later trials. Cand2's original search considered CV and chronological performance through the combined score J. It outperformed anchored on J's chronological checks, although this advantage did not hold on our stricter chronology benchmark. That mixed evidence motivated trying a small Cand2 contribution while keeping most of anchored's predictions. The blend maintained similar local performance and subsequently scored .89935 on Kaggle, compared with .89850 for anchored/LDA, so we retained it as our final model.

On the 78-row validation set, the selected blend predicts the same labels as anchored. Its per-reason F1 is .941 for Safety (9 positives), .667 for Funding (4), .759 for Efficacy (14), .990 for Recruitment (50), and .471 for Other (10). Overall, sample-wise F1 is .891 and macro F1 is .765. These validation results reflect our development split; the Kaggle column gives the leaderboard scores recorded for our submissions. The repeated splits overlap, so these results do not establish statistical significance.

2.2. Part 2: outcome prediction

Table 2 summarizes our Part 2 results; lower log loss is better. We selected file 13. File 15 improved both local scores but performed worse on Kaggle, showing that these local gains did not carry over.

2.2.1. Validation by phase

Table 3 compares the same held-out validation rows. Relative to the baseline, our final model improves ROC-AUC in all three phases and F1 in Phases I and III, while Phase II F1 falls from .657 to .647. Phase III log loss improves from .529 to .513, but Phase I rises from .556 to .559.

Model	Train	Pooled	Val.	Time	Kaggle
80/20 Cand2	**.9352**	.9216	.891	**.888**	**.89935**
75/25 LDA	.9321	**.9234**	.891	.884	.89850
Anchored	.9321	.9214	.891	.884	.89636
LDA ensemble	.9318	.9216	**.906**	.878	.89401
20% PySR	.9275	.9198	.885	.878	.89423
20% kernel	.9334	.9209	.887	.871	.89316

Table 1. Part 1 sample-wise F1, higher is better. Train and pooled average eight grouped repeats on 157 and 235 labels; validation uses 78 trials; time pools three windows. Cand2, LDA, and kernel rows blend with anchored. PySR's parent is the earlier 75/25 anchored/LDA blend.

Model	Time	Grouped	Kaggle
Phase-specific ensemble	—	—	0.59562
Text-heavy ensemble	—	—	0.58993
Stack, no recency	0.5421	0.5797	0.58810
Recency stack	0.5374	0.5874	0.58688
File 10, recent neighbors	0.5359	0.5869	0.58609
File 13, 30% early ensemble	0.5334	0.5790	**0.58465**
File 15, tree and trend	**0.5327**	**0.5783**	0.58492

Table 2. Part 2 log loss. Time averages the forward windows. Grouped is trial-grouped cross-validation. The first two submissions used an earlier local protocol, so those cells are blank.

Thus the modification has mixed effects even though the phase-averaged log loss improves. The final model's phase-averaged validation log loss is .55328, compared with .58465 on Kaggle, a gap of .03137 consistent with the later test cohort being harder to predict. File 10 also illustrates the selection trade-off: it is weaker than the recency stack on random validation but stronger on the forward windows and Kaggle.

2.2.2. Errors by stopping reason

Table 4 compares the baseline and final model using the same reason groups. False-success rates fall for Phase I Recruitment (.36 to .09) and Phase II Recruitment (.20 to .14), and for trials without a recorded reason in Phases I and II. Phase III remains difficult: Recruitment errors increase from .52 to .57, while the no-reason rate stays at .56. Efficacy errors fall from 8/9 to 7/9, but all nine reason tags are predictions. Small groups also move in opposite directions: Phase I Safety improves from 5/9 to 3/9, while Efficacy worsens from 1/2 to 2/2. These small counts make category-specific gains uncertain.

Phase	Metric	Base	Recency	File 10	Final
I	F1	0.765	**0.793**	0.762	0.788
	ROC-AUC	0.783	**0.794**	0.787	0.791
	PR-AUC	**0.835**	0.830	0.824	0.832
	log loss	0.556	0.556	0.560	0.559
II	F1	**0.657**	0.647	0.638	0.647
	ROC-AUC	0.740	0.753	0.754	**0.757**
	PR-AUC	0.717	0.732	0.730	**0.735**
	log loss	0.600	0.593	0.593	**0.587**
III	F1	0.833	0.825	0.829	**0.839**
	ROC-AUC	0.774	0.790	0.786	**0.795**
	PR-AUC	0.862	0.882	0.877	**0.883**
	log loss	0.529	0.516	0.521	**0.513**

Table 3. Part 2 held-out validation metrics. Final is file 13; Base is the eligibility logistic model. F1 uses a 0.5 cutoff; PR-AUC is average precision. Phase sizes are 149, 501, and 363. Best value per row is bold; ties at the displayed precision are unbolded.

Phase	Reason	n	Obs./Pred.	Base	Final
I	Safety	9	1/8	.56	.33
I	Funding	4	1/3	.50	.25
I	Efficacy	2	0/2	.50	1.00
I	Recruitment	22	2/20	.36	.09
I	Other	5	0/5	.80	.40
I	No reason	28	—	.43	.32
II	Safety	15	1/14	.40	.33
II	Funding	4	1/3	.00	.00
II	Efficacy	10	0/10	.30	.30
II	Recruitment	87	12/75	.20	.14
II	Other	18	3/15	.33	.22
II	No reason	144	—	.35	.28
III	Safety	6	1/5	.50	.50
III	Funding	0	0/0	—	—
III	Efficacy	9	0/9	.89	.78
III	Recruitment	21	5/16	.52	.57
III	Other	8	2/6	.63	.63
III	No reason	82	—	.56	.56

Table 4. False-success rates among failed validation trials. Obs./Pred. counts observed and predicted reason tags within each group. "No reason" means absent from Part 1. Multi-label trials enter each applicable reason.

3. Discussion

3.1. Part 1: what changed our approach

We began with logistic regression trained only on the observed labels. This gave us a reference for deciding whether semi-supervised learning actually helped, rather than assuming that adding generated labels would improve the model. The first k-means experiments showed the risk: their generated labels mostly reinforced Recruitment and

hurt validation performance, especially for rarer reasons. We learned to pay attention to which labels we were adding, not just how many trials we could add. This motivated more conservative label generation and the combination of supervised scores with clustering.

Our evaluation procedure changed as we experimented. Initially, we relied heavily on the supplied validation set, but repeatedly choosing settings on just 78 trials could favor quirks of that split. We moved to repeated CV on both the 157 training labels and the pooled 235 training-plus-validation labels. The first kept the original training setting as a reference; the second let each fold fit on more observed labels while still holding out the trials being evaluated. Looking for gains across both pools gave us a more stable basis for tuning than repeatedly chasing the best validation score. These were still development comparisons, not independent confirmation.

We also tested whether the available clinical and drug information could add to the embeddings. We added eligibility text, disease and drug information, pathways, molecular features, and year to the strongest models at that stage, but the feature screens did not improve their primary score. We therefore returned to embeddings as our main inputs. Our interpretation is that the embeddings already describe the stopping reason directly, while the other fields describe broader clinical context. With few observed labels, extra sparse dimensions and missing drug annotations could add noise more easily than useful information. These experiments supported a simpler representation, although we did not repeat every feature screen on the final blend.

Random CV did not address our concern about predicting future trials: the test trials started later, and the mix of stopping reasons could change over time. We added earlier-to-later checks to see how models behaved under that shift. They gave us another benchmark, but did not consistently identify the best Kaggle candidate. We also investigated the combined evaluation score J, defined in the benchmark notebook as $J = (A + B)/2$. Here A is ten-repeat random CV on 235 labels, and B is the mean of three chronological checks. This experiment used different split boundaries from our grouped benchmark and aimed to rank models by broader generalization. Its rankings sometimes disagreed with Kaggle. We learned that averaging two plausible checks does not automatically produce a dependable selection metric; we kept chronology as a separate diagnostic rather than relying on J alone.

The per-reason results also showed that a strong overall score could hide weaknesses in Funding and Other. We tried extra weighting for those labels and nonlinear specialists for Other, hoping that a focused model could recover cases the shared classifier missed. The weighting changes did not improve decisions, and the specialists did not give a reliable overall gain. We also found that almost all generated examples had only one positive reason, suggesting that self-training could reinforce missed secondary labels. Leaving unselected Other labels unknown recovered more true Other cases, but also introduced enough false positives to reduce overall F1. The lesson was that improving recall for a difficult reason was insufficient if precision fell too far. Funding remained especially hard to assess with so few examples.

These findings shaped our blending strategy. We kept most of the weight on the stronger anchored model and used LDA or Cand2 to contribute different prediction patterns. Cand2's weaker standalone result did not rule out a useful contribution, and its small share improved the final blend. However, learned combiners, explicit diversity penalties, log-odds pooling, and symbolic regression did not improve our selected result. Our strongest model combined different approaches without moving far from the model that already worked well.

3.2. Part 2: temporal shift and error trade-offs

In Part 2, better log loss did not always mean better F1: a probability can become more accurate while crossing the 0.5 threshold in a way that lowers F1. Because the test trials start later and success rates rose within our labeled data, we prioritized chronological evaluation. Recency weighting and recent neighbors reduce the influence of older trials. The earlier ensemble helped through different error patterns, although increasing its share from 20% to 30% improved Kaggle log loss by only 0.00009. File 15's worse leaderboard result reinforced the limits of historical averages.

Our error analysis suggests that Phase III failures are particularly difficult. These trials may reach completion but miss their endpoints, leaving eligibility text similar to that of successful trials. Recruitment and funding failures may reflect administrative constraints, and the ensemble identifies some additional cases. This motivated adding clinical context beyond eligibility text, but the final model still trades improvements in some groups for errors in others. These explanations remain tentative: Safety, Funding, and Other have few examples in each phase, and predicted reason tags can place trials in the wrong category. The no-reason groups contain trials absent from Part 1, rather than a predicted "no reason" category.

AI disclosure. We used AI tools to assist with model improvements, resolve errors, automate testing, and edit and format this report.

References

[1] Miles Cranmer. Interpretable machine learning for science with PySR and SymbolicRegression.jl. *arXiv preprint arXiv:2305.01582*, 2023. 2

[2] Arthur P. Dempster, Nan M. Laird, and Donald B. Rubin. Maximum likelihood from incomplete data via the EM algo-

rithm. *Journal of the Royal Statistical Society: Series B*, 39 (1):1–22, 1977. 1

[3] Tianfan Fu, Kexin Huang, Cao Xiao, Lucas M. Glass, and Jimeng Sun. Hint: Hierarchical interaction network for clinical-trial-outcome predictions. *Patterns*, 3(4):100445, 2022. 1

[4] Qian Huang, Horace He, Abhay Singh, Ser-Nam Lim, and Austin R. Benson. Combining label propagation and simple models out-performs graph neural networks. *arXiv preprint arXiv:2010.13993*, 2020. 2

[5] Greg Landrum et al. RDKit: Open-source cheminformatics. https://www.rdkit.org. Accessed November 2026. 1

[6] Marija Milacic, Deidre Beavers, Patrick Conley, et al. The Reactome pathway knowledgebase 2024. *Nucleic Acids Research*, 52(D1):D672–D678, 2024. 1

[7] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011. 1

[8] Liudmila Prokhorenkova, Gleb Gusev, Aleksandr Vorobev, Anna Veronika Dorogush, and Andrey Gulin. CatBoost: Unbiased boosting with categorical features. *arXiv preprint arXiv:1706.09516*, 2017. 3

[9] David Rogers and Mathew Hahn. Extended-connectivity fingerprints. *Journal of Chemical Information and Modeling*, 50(5):742–754, 2010. 1

[10] Isaac Triguero, Salvador García, and Francisco Herrera. Self-labeled techniques for semi-supervised learning: taxonomy, software and empirical study. *Knowledge and Information Systems*, 42(2):245–284, 2015. 2

[11] U.S. National Library of Medicine. ClinicalTrials.gov. https://clinicaltrials.gov. Accessed November 2026. 1

[12] David H. Wolpert. Stacked generalization. *Neural Networks*, 5(2):241–259, 1992. 2

[13] David Yarowsky. Unsupervised word sense disambiguation rivaling supervised methods. In *Proceedings of the 33rd Annual Meeting of the Association for Computational Linguistics*, pages 189–196, 1995. 2